

Task 1: Find Max Value in an Array

ALGORITHM FindMax($A[a1..an]$, k)

// Finds the maximum value in an array

// Input: Array $A[a1..an]$, num k

// Output: Maximum value in A

$\text{max} \leftarrow A[0]$

for $i \leftarrow 1$ to n do

 if $A[i] > \text{max}$ then

$\text{max} \leftarrow A[i]$

return max

Task 2: Calculate Average of Array Numbers

ALGORITHM CalculateAverage($A[a1..an]$, k)

// Calculates the average of an array's numbers

// Input: Array $A[a1..an]$, num k

// Output: Average value of A

$\text{sum} \leftarrow 0$

for $i \leftarrow 1$ to k do

$\text{sum} \leftarrow \text{sum} + A[i]$

$\text{average} \leftarrow \text{sum} / k$

return average

1.1.4

ALGORITHM SquareRoot(n)

// Computes the square root of a positive integer n

// Input: A positive integer n

// Output: An approximation of $\sqrt{n}$

$x \leftarrow n$

$y \leftarrow (x + n / x) / 2$

while $y < x$ do

$x \leftarrow y$

$y \leftarrow (x + n / x) / 2$

return x

1.1.6

a. Euclidean Algorithm Iteration Steps for $\gcd(31415, 14142)$:

- Step 1: $(31415, 14142) \rightarrow 31415 \bmod 14142 = 3131$
- Step 2: $(14142, 3131) \rightarrow 14142 \bmod 3131 = 1618$
- Step 3: $(3131, 1618) \rightarrow 3131 \bmod 1618 = 1513$
- Step 4: $(1618, 1513) \rightarrow 1618 \bmod 1513 = 105$
- Step 5: $(1513, 105) \rightarrow 1513 \bmod 105 = 43$
- Step 6: $(105, 43) \rightarrow 105 \bmod 43 = 19$
- Step 7: $(43, 19) \rightarrow 43 \bmod 19 = 5$
- Step 8: $(19, 5) \rightarrow 19 \bmod 5 = 4$
- Step 9: $(5, 4) \rightarrow 5 \bmod 4 = 1$
- Step 10: $(4, 1) \rightarrow 4 \bmod 1 = 0$

Answer: $\gcd(31415, 14142) = 1$. Euclid's algorithm takes 10 division steps.

b. Speedup Estimation:

The consecutive integer checking algorithm starts checking from $t = \min(31415, 14142) = 14142$ down to 1. Since $\gcd = 1$, it checks all 14,142 values and performs at least 14,142 division operations.

- Consecutive checking steps = 14,142

- Euclid's algorithm steps = 10

- Estimated speedup ratio = $14142 / 10 \approx 1414$ times faster.

1.1.7 Proof of GCD Property

Prove: $\gcd(m, n) = \gcd(n, m \bmod n)$ for all positive integers m and n .

Proof:

Let $r = m \bmod n$. By definition of integer division, $m = q * n + r$ for some integer q .

1) If d divides both m and n , then d must also divide $r = m - q * n$. Thus, d is a common divisor of n and r .

2) If d divides both n and r , then d must also divide $m = q * n + r$. Thus, d is a common divisor of m and n .

Since the pairs (m, n) and (n, r) share the exact same set of common divisors, their greatest common divisors are equal:

$\gcd(m, n) = \gcd(n, m \bmod n)$.

1.1.8 Behavior when $m < n$

If $m < n$, the modulo operation gives $m \bmod n = m$.

In the very first iteration, the algorithm assigns:

- $r \leftarrow m \bmod n$ (which equals m)

- $m \leftarrow n$

- $n \leftarrow r$ (which equals original m)

Thus, it automatically swaps the inputs $(m, n) \rightarrow (n, m)$ in 1 extra step. After this first step, m is guaranteed to be strictly greater than n for all subsequent iterations.

1.1.9 Modulo Steps Bounds for $1 \leq m, n \leq 10$

a. Minimum number of steps: 1 step.

Occurs when n divides m (e.g., $m = 10, n = 5$, where $10 \bmod 5 = 0$).

b. Maximum number of steps: 5 steps.

Occurs for consecutive Fibonacci numbers, such as $m = 5, n = 8$:

```
-----  
m | n | r = m mod n  
-----  
5 | 8 | 5  
8 | 5 | 3  
5 | 3 | 2  
3 | 2 | 1  
2 | 1 | 0
```

1.2.4

ALGORITHM QuadraticRoots(a, b, c)

// Finds the real roots of a quadratic equation $ax^2 + bx + c = 0$

// Input: Real coefficients a, b , and c

// Output: The real roots of the equation (if any)

if $a = 0$ then

 if $b = 0$ then

 if $c = 0$ then

 return "Infinitely many roots"

 else

 return "No roots"

 else

 return $-c / b$

else

$D \leftarrow b^2 - 4 * a * c$

 if $D < 0$ then

 return "No real roots"

 else if $D = 0$ then

 return $-b / (2 * a)$

 else

$x1 \leftarrow (-b + \text{sqrt}(D)) / (2 * a)$

$x2 \leftarrow (-b - \text{sqrt}(D)) / (2 * a)$

 return $x1, x2$

1.2.5 Decimal to Binary Conversion

a. Algorithm Description:

1. Divide the decimal number n by 2 and record the remainder (0 or 1).
2. Update n with the integer quotient $\lfloor n / 2 \rfloor$.
3. Repeat steps 1–2 until n becomes 0.
4. Read the recorded remainders in reverse order (from last to first) to form the binary representation.

b. Algorithm Pseudocode:

```
ALGORITHM BinaryRepresentation(n)
// Converts a positive decimal integer n into its binary representation
// Input: A positive integer n
// Output: The binary representation of n as a string of bits
bits ← ""
while n > 0 do
    r ← n mod 2
    bits ← r + bits // prepend the remainder
    n ← ⌊n / 2⌋
return bits
```

1.2.6

```
ALGORITHM ATMDispenseCash(card, pin, amount)
```

```
// Dispenses cash from an ATM
```

```
// Input: Card, PIN, and requested amount
```

```
// Output: Cash or an error message
```

```
Insert card
```

```
if card is valid then
```

```
    Enter PIN
```

```
    if PIN is correct then
```

```
        Enter amount
```

```
        if amount ≤ account balance then
```

```
            if amount ≤ ATM cash available then
```

```
                Dispense cash
```

```
                Update account balance
```

```
                return "Transaction successful"
```

```
            else
```

```
                return "Insufficient cash in ATM"
```

```
        else
```

```
            return "Insufficient account balance"
```

```
    else
```

```
        return "Incorrect PIN"
```

```
else
```

```
    return "Invalid card"
```

1.2.9

ALGORITHM MinDistance($A[0..n-1]$)

// Finds the minimum distance between two elements in an array

// Input: Array $A[0..n-1]$ of numbers

// Output: Minimum distance between two of its elements

$dmin \leftarrow \infty$

for $i \leftarrow 0$ to $n - 2$ do

 for $j \leftarrow i + 1$ to $n - 1$ do

$temp \leftarrow |A[i] - A[j]|$

 if $temp < dmin$ then

$dmin \leftarrow temp$

return $dmin$

Theoretical Task 1: Analysis of Sieve of Eratosthenes Properties

Question: Prove that if a positive composite integer n has no prime factors less than or equal to $\sqrt{n}$, then n must be a prime number.

Proof: Assume n is a composite number. By definition, n can be factored into two integers a and b such that $1 < a \leq b < n$, meaning $n = a * b$.

Suppose both $a > \sqrt{n}$ and $b > \sqrt{n}$.

Multiplying the two inequalities gives $a * b > \sqrt{n} * \sqrt{n} = n$, which contradicts $n = a * b$.

Thus, at least one factor (say a) must satisfy $a \leq \sqrt{n}$.

By the Fundamental Theorem of Arithmetic, a itself must have at least one prime factor $p \leq a \leq \sqrt{n}$.

Therefore, if n is composite, it must have a prime factor $p \leq \sqrt{n}$. By contrapositive, if n has no prime factors $p \leq \sqrt{n}$, n must be prime.
(Q.E.D.)

Theoretical Task 2: Worst-Case Analysis of Sequential Search

Question: Analyze the exact number of key comparisons made by Sequential Search in the worst-case scenario and when the search key is not present in an array of size n .

Analysis: Consider searching for a target key K in an array $A[0..n-1]$.

Worst-Case (Key at the last position $A[n-1]$): The algorithm compares K with $A[0]$, $A[1]$, ..., $A[n-1]$ sequentially. It makes 1 comparison per element, reaching the last index $n-1$. Total key comparisons = n .

Unsuccessful Search (Key not in A): The algorithm scans through every element from index 0 to $n-1$ without finding a match. After checking all n elements, the search terminates. Total key comparisons = n .

Conclusion: In both cases, the exact number of key comparisons is n , giving a worst-case time complexity of $O(n)$ (or $\Theta(n)$).

Practical Task 1: Count Even Numbers in an Array

ALGORITHM CountEvenNumbers($A[0..n-1]$)

// Counts the number of even elements in a given array

// Input: An array $A[0..n-1]$ of integers

// Output: The total count of even numbers in A

count $\leftarrow$ 0

for $i \leftarrow 0$ to $n - 1$ do

if $A[i] \bmod 2 = 0$ then

count $\leftarrow$ count + 1

return count

Practical Task 2: Linear Search for Target Element

ALGORITHM LinearSearch($A[0..n-1]$, K)

// Searches for a given key K in an array A

// Input: An array $A[0..n-1]$ and a search key K

// Output: The index of the first occurrence of K in A , or -1 if not found

for $i \leftarrow 0$ to $n - 1$ do

if $A[i] = K$ then

return i

return -1